

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

Aim: Adaboost Ensemble Learning

Implement the Adaboost algorithm to create an ensemble of weak classifiers.

Train the ensemble model on a given dataset and evaluate its performance.

Compare the results with individual weak classifiers.

Input

```
PRACTICAL 6.py - C:/Users/DELL/Downloads/PRACTICAL 6.py (3.13.11)
File Edit Format Run Options Window Help

# AdaBoost Ensemble Learning

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# Load dataset
data = pd.read_csv("luxury_cosmetics_fraud_analysis_2025.csv")

print("Dataset loaded successfully")
print("Dataset shape:", data.shape)

print("InColumns:")
print(data.columns)

# Target column
target = "Fraud_Flag"

# Remove missing target values
data = data.dropna(subset=[target])

# Convert categorical columns into numbers
for column in data.columns:
    if data[column].dtype == "object":
        encoder = LabelEncoder()
        data[column] = encoder.fit_transform(data[column].astype(str))

# Separate fraud and normal transactions
normal = data[data[target] == 0]
fraud = data[data[target] == 1]

print("InOriginal data:")
print("Normal transactions:", len(normal))
print("Fraud transactions:", len(fraud))

# Balance dataset
number = min(len(normal), len(fraud))

normal = normal.sample(
    n=number,
    random_state=42
)

fraud = fraud.sample(
    n=number,
    random_state=42
)

data = pd.concat([normal, fraud])

# Shuffle data
data = data.sample(
    frac=1,
    random_state=42
).reset_index(drop=True)

print("InBalanced data:")
print("Normal transactions:", sum(data[target] == 0))
print("Fraud transactions:", sum(data[target] == 1))

# Separate features and target
X = data.drop(target, axis=1)
y = data[target].astype(int)

# Keep numerical features
X = X.select_dtypes(include=["number"])
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
PRACTICAL 6.py - C:/Users/DELL/Downloads/PRACTICAL 6.py (3.13.11)
File Edit Format Run Options Window Help
X = X.select_dtypes(include=["number"])

# Replace missing values
X = X.fillna(0)

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("nTraining records:", len(X_train))
print("nTesting records:", len(X_test))

# Decision Stump
weak_classifier = DecisionTreeClassifier(
    max_depth=1,
    random_state=42
)

print("nTraining Decision Stump...")

weak_classifier.fit(X_train, y_train)

weak_prediction = weak_classifier.predict(X_test)

weak_accuracy = accuracy_score(
    y_test,
    weak_prediction
)

print("Decision Stump Accuracy:")
print(round(weak_accuracy * 100, 2), "%")

Ln: 225 Col: 0

PRACTICAL 6.py - C:/Users/DELL/Downloads/PRACTICAL 6.py (3.13.11)
File Edit Format Run Options Window Help

# AdaBoost
adaboost = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(
        max_depth=1,
        random_state=42
    ),
    n_estimators=50,
    learning_rate=1.0,
    random_state=42
)

print("nTraining AdaBoost...")

adaboost.fit(X_train, y_train)

adaboost_prediction = adaboost.predict(X_test)

adaboost_accuracy = accuracy_score(
    y_test,
    adaboost_prediction
)

print("AdaBoost Accuracy:")
print(round(adaboost_accuracy * 100, 2), "%")

# Decision Stump confusion matrix
print("nDecision Stump Confusion Matrix:")

print(
    confusion_matrix(
        y_test,
        weak_prediction
    )
)

# AdaBoost confusion matrix
print("nAdaBoost Confusion Matrix:")

Ln: 225 Col: 0
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
PRACTICAL 6.py - C:/Users/DELL/Downloads/PRACTICAL 6.py (3.13.11)
File Edit Format Run Options Window Help

print("\nAdaBoost Confusion Matrix:")

cm = confusion_matrix(
    y_test,
    adaboost_prediction
)

print(cm)

# AdaBoost classification report
print("\nAdaBoost Classification Report:")

print(
    classification_report(
        y_test,
        adaboost_prediction,
        target_names=["No Fraud", "Fraud"],
        zero_division=0
    )
)

# Accuracy comparison graph
models = [
    "Decision Stump",
    "AdaBoost"
]

accuracies = [
    weak_accuracy * 100,
    adaboost_accuracy * 100
]

plt.bar(
    models,
    accuracies
)

plt.title("Decision Stump vs AdaBoost")
plt.xlabel("Model")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 100)
plt.show()

# Confusion matrix graph
plt.imshow(cm)

plt.title("AdaBoost Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(
    [0, 1],
    ["No Fraud", "Fraud"]
)

plt.yticks(
    [0, 1],
    ["No Fraud", "Fraud"]
)

for i in range(2):
    for j in range(2):
        plt.text(
            i,
            j,
            cm[i, j],
            ha="center"
        )

L: 225 Col: 0
```

```
PRACTICAL 6.py - C:/Users/DELL/Downloads/PRACTICAL 6.py (3.13.11)
File Edit Format Run Options Window Help

plt.title("Decision Stump vs AdaBoost")
plt.xlabel("Model")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 100)
plt.show()

# Confusion matrix graph
plt.imshow(cm)

plt.title("AdaBoost Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(
    [0, 1],
    ["No Fraud", "Fraud"]
)

plt.yticks(
    [0, 1],
    ["No Fraud", "Fraud"]
)

for i in range(2):
    for j in range(2):
        plt.text(
            i,
            j,
            cm[i, j],
            ha="center"
        )

L: 162 Col: 0
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
)  
plt.yticks(  
    [0, 1],  
    ["No Fraud", "Fraud"]  
)  
for i in range(2):  
    for j in range(2):  
        plt.text(  
            j,  
            i,  
            cm[i, j],  
            ha="center",  
            va="center"  
        )  
plt.colorbar()  
plt.show()
```

Output

```
IDLE Shell 3.13.11  
File Edit Shell Debug Options Window Help  
Dataset loaded successfully  
Dataset shape: (2133, 16)  
  
Columns:  
Index(['Transaction_ID', 'Customer_ID', 'Transaction_Date', 'Transaction_Time',  
       'Customer_Age', 'Customer_Loyalty_Tier', 'Location', 'Store_ID',  
       'Product_SKU', 'Product_Category', 'Purchase_Amount', 'Payment_Method',  
       'Device_Type', 'IP_Address', 'Fraud_Flag', 'Footfall_Count'],  
      dtype='object')  
  
Original data:  
Normal transactions: 2067  
Fraud transactions: 66  
  
Balanced data:  
Normal transactions: 66  
Fraud transactions: 66  
  
Training records: 105  
Testing records: 27  
  
Training Decision Stump...  
Decision Stump Accuracy:  
44.44 %  
  
Training AdaBoost...  
AdaBoost Accuracy:  
40.74 %  
  
Decision Stump Confusion Matrix:  
[[ 2 12]  
 [ 3 10]]  
  
AdaBoost Confusion Matrix:  
[[6 8]  
 [8 5]]
```

```
>>>  
AdaBoost Confusion Matrix:  
[[6 8]  
 [8 5]]  
  
AdaBoost Classification Report:  
precision recall f1-score support  
  
No Fraud 0.43 0.43 0.43 14  
Fraud 0.38 0.38 0.38 13  
  
accuracy 0.41 27  
macro avg 0.41 0.41 0.41 27  
weighted avg 0.41 0.41 0.41 27
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

Figure 1

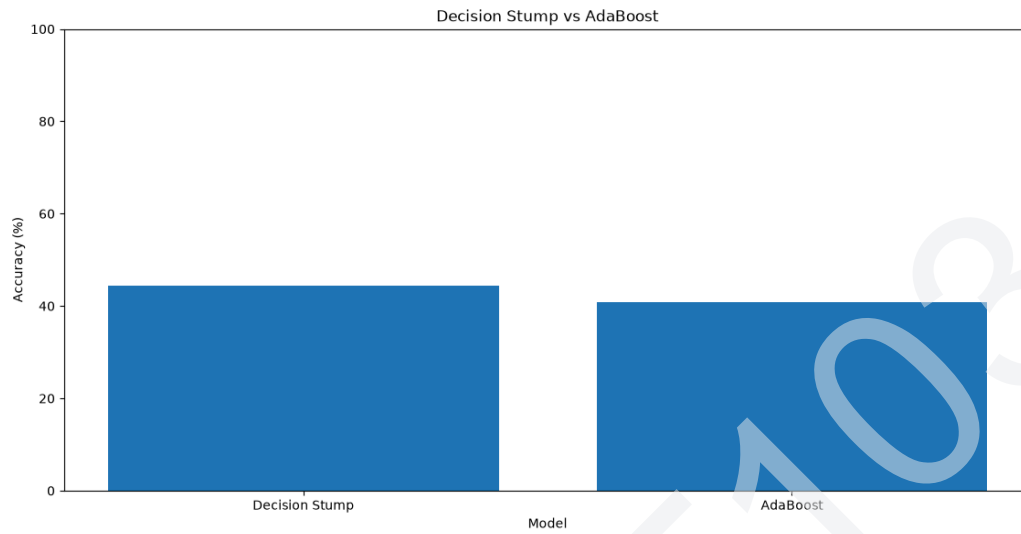


Figure 1

